

Git and Source Code Version Control

A content-addressed object graph, and a way to move pointers into it. Branching strategy, review and delivery practice are your team's policy.

Ondrej (Ondra) Krajicek

me@ondrejkrjicek.com · www.linkedin.com/in/OndrejKrajicek

v1 · 4e9f6b455 · built 2026-10-08

Generated by the agentic vault — an autonomous research and authoring harness. Every factual claim was gated against a cited source registry before this document was produced.

Three messages you have already seen

Each one is an exact statement about a small data structure. None of them is an opinion.

- 1 Push refused** `! [rejected] main → main (non-fast-forward)`
- 2 Pull refused** `fatal: Need to specify how to reconcile
divergent branches.`
- 3 A warning after checkout** `You are in 'detached HEAD' state.`

Today these read as obstacles. By the end they read as reports about pointers.

Roadmap

Eight parts. The model first, team practice second, a decision at each end.

I · What a commit is	Four object types, by hand.
II · Pointers and the graph	Three ways to combine two branches.
III · Centralisation moves	Out of the data model.
IV · What Git costs	Rival designs.
V · Branching strategy	One variable.
VI · Review	What measurements show.
VII · Delivery practice	Three things, one name.
VIII · What stays open	What is not reached.

Takeaways

- | | | |
|----------|----------------------------------|--|
| 1 | Objects and pointers | A commit is a snapshot. A branch is a pointer. |
| 2 | One merge function | Three ways to combine two branches, three graphs. |
| 3 | Trade-offs, not laws | Every Git design choice had an alternative, and some system took it. |
| 4 | Branch lifetime | A contract about how long work stays unmerged. |
| 5 | What review produces | Understanding first, defects second, none found after 90 minutes. |
| 6 | Three practices, one name | Each one needs a different branching strategy. |

SCOPE

What is here, and what is not

COVERED

- The object model, rebuilt with low-level commands.
- Three ways to combine two branches, and one remote.
- Branching strategy, review and delivery practice.
- Two systems that made the opposite design choice.

NOT COVERED

- Any graphical client. The command line only.
- Any hosting service's wider interface.
- Interactive rebase corner cases, submodules, `git worktree`.
- Multiple remotes and forks.

PART I

What a commit actually is

Four object types, one kind of pointer, and a repository rebuilt by hand.

Git is a filesystem with a version-control interface on top

Content addressing: store bytes, get back a name derived from those bytes.

Give Git a file. It writes the bytes into `.git/objects/` under a name that is the hash of a short header plus the content. Ask for that name later and the same bytes come back.

Identical content always produces the same name. Two files with the same bytes are stored once, in every repository that holds them.

Four object types, and nothing else

TYPE	WHAT IT HOLDS
<code>blob</code>	File content. No filename and no path.
<code>tree</code>	Directory entries. A mode, a type, a name and a hash each.
<code>commit</code>	One tree hash, zero or more parent hashes, an author and a message.
<code>tag</code>	A target hash, a tagger and a message.

A commit hash covers its tree hash and its parent hashes. A tree hash covers the hashes below it. So one commit hash names the whole project state and all history behind it.

Open .git and look

A fresh repository is one line of text, a directory of small files, and an empty object store.

```
$ git init demo && cd demo
$ ls .git
HEAD config description hooks info objects refs
$ cat .git/HEAD
ref: refs/heads/main
$ find .git/objects -type f
```

HEAD names the commit you have checked out. Here it does so indirectly, by naming a branch. refs is a directory of small files, and objects is empty, because nothing has been stored yet.

A blob is content without a name

Two low-level commands write one object and read it back.

```
$ printf 'test content\n' | git hash-object -w --stdin
d670460b4b4aece5915caf5c68d12f560a9fe3e4
$ find .git/objects -type f
.git/objects/d6/70460b4b4aece5915caf5c68d12f560a9fe3e4
$ git cat-file -t d670460
blob
$ git cat-file -p d670460
test content
```

The first two characters of the hash are the directory. The remaining thirty-eight are the filename. Nowhere in that object is a filename recorded.

Low-level commands like these two are called plumbing. The everyday commands you already type are called porcelain.

The index is the third state

Between the files on disk and a commit sits one file that maps every tracked path to a hash.

The working tree is the checked-out files you can edit. The index is `.git/index`, a list of paths with the hash of the content staged for each.

`git add` computes a blob and writes the path and hash into the index. Nothing is committed yet.

A tree gives content its names

Write an entry into the index, then turn the index into a tree object.

```
$ git update-index --add --cacheinfo 100644 \  
    d670460b4b4aece5915caf5c68d12f560a9fe3e4 test.txt  
$ git write-tree  
80865964295ae2f11d27383e5f9c0b58a8ef21da  
$ git cat-file -p 8086596  
100644 blob d670460b4b4aece5915caf5c68d12f560a9fe3e4    test.txt
```

`--cacheinfo` writes an index entry directly, from a mode, a hash and a path, with no file on disk. 100644 is the mode: Git records only normal file, executable file and symbolic link, and no owner, group or permission bits.

A commit is a tree, its parents and a message

One more command wraps the tree. The result is a snapshot with provenance.

```
$ echo 'First commit' | git commit-tree 8086596
e0437810144e64a708a6441a7094db56fe376eda
$ git cat-file -p e043781
tree 80865964295ae2f11d27383e5f9c0b58a8ef21da
author Student <s@example.org> 1774000000 +0100
First commit
```

A first commit has no parent line. Every later commit records the hash of the commit it was built on. That single link is what makes history a graph.

Your own commit hash will differ. The author name and the timestamp are hashed too.

A ref is a file with a hash in it

Objects are immutable. The only mutable thing in a repository is a pointer.

```
$ git update-ref refs/heads/main e043781
$ cat .git/refs/heads/main
e0437810144e64a708a6441a7094db56fe376eda
$ cat .git/HEAD
ref: refs/heads/main
```

A branch is that file. Creating one writes forty-one bytes. Deleting one removes a file. Nothing is copied, at any repository size.

Predict: what did those five commands build?

Write your answer down before anything else runs. Answer with equalities, not with hex.

- | | | |
|----------|-----------------------------|---|
| 1 | Object count | How many files are under <code>.git/objects</code> now? |
| 2 | Porcelain equivalent | Which everyday command does each of the five replace? |
| 3 | Second commit | Commit again with nothing changed, using <code>git commit --allow-empty</code> . Which hashes change? |

Observe

```
$ find .git/objects -type f | wc -l
3
$ git log --oneline
e043781 First commit
$ git cat-file -p HEAD | head -1
tree 80865964295ae2f11d27383e5f9c0b58a8ef21da

$ git commit --allow-empty -qm 'Second commit'
$ git cat-file -p HEAD | head -2
tree 80865964295ae2f11d27383e5f9c0b58a8ef21da
parent e0437810144e64a708a6441a7094db56fe376eda
```

Three objects: one blob, one tree, one commit. The second commit carries the same tree hash and a new commit hash.

Explain: add, commit and branch were these all along

git add

hash-object -w plus an index entry.

git commit

write-tree, then commit-tree, then move the branch file.

git branch

update-ref. One small file, one hash.

The five commands added objects and moved one pointer. None of them edited an object in place. That is the whole vocabulary of the tool.

Why `.git` is not enormous

Every commit holds the whole project, and the cost tracks only what changed.

A file that did not change hashes to the same blob, so the new tree points at the object that already exists. A directory that did not change also hashes to the same tree. Git therefore shares everything below it instead of copying it.

This is the mechanism behind container image layers. Each party sees what looks like a private copy. Both parties point at one physical original.

Snapshots are the model, packfiles are the storage

Both halves are true. A delta is a recipe for rebuilding one object from another plus the difference.

THE MODEL

A commit names a whole tree.
There is no delta anywhere in the object graph.

THE STORAGE LAYER

Git compresses each object on its own as it writes it. Git also packs many objects into one file from time to time. Only inside a packfile does Git store objects as deltas against one another.

The same two lines can hash to two values

Content addressing hashes bytes. Line endings are bytes.

A file saved with carriage-return and line-feed endings is not byte-identical to the same text saved with line-feed endings. The hashes differ.

Git normalises line endings to line feed in the index when a file is checked in. The default on Windows checks files out with carriage returns. So two laptops agree on the hash only when the file reaches the index with the same bytes.

The hash detects accidents

Tamper-evidence is structural. Adversary resistance is a property of the hash function.

Change one byte below a commit and the commit hash changes. Two people who agree on a hash agree on the whole history behind it. An attack finds a collision: two inputs, one hash.

Git's own breaking-changes document names four attacks on the SHA-1 default. They are the SHAppening in 2015, SHAttered in 2017, a birthday-near-collision in 2019, and Shambles in 2020.

The default moves to SHA-256 for new repositories at version 3.0. News reporting of a maintainer summit gives April 2027 for that release. The project's own document carries no date.

Add objects, or move refs

Two operations. Every command in the tool is one of them, or both.

ADDS OBJECTS

add, commit, fetch, merge.
Additive, and safe on shared work.

MOVES REFS

branch, reset, checkout, push. A ref that moves backwards unnames work.

commit --amend, rebase and cherry-pick do both. Cherry-pick copies one commit onto another branch. All three build new objects and move a pointer, and none of them modifies the commit you started from.

Pointers, the graph, and one remote

Merge, rebase and squash are one function. A remote adds one more namespace of pointers.

A branch is a pointer, not a container

Nothing is inside a branch. A branch names one commit, and that commit names its parents.

Switching branches rewrites HEAD and replaces the files in the working tree. It does not move you into anything.

A branch owns no commits. A commit is reachable from a branch, or it is not. Git computes reachability by walking parent links.

Ancestor, and the merge base

Two words carry every rule in this part. Both are defined by walking parent links.

WORD	DEFINITION
ancestor	Commit A is an ancestor of B when A is reached by following B's parent links.
merge base	A common ancestor of two commits with no better common ancestor below it.

A merge base is not "where the branches split". The ancestry ordering defines it. Git states plainly that a pair of commits can have more than one.

A fast-forward creates nothing

When one side is an ancestor of the other there is nothing to reconcile.

If your branch tip is an ancestor of the other branch tip, Git moves your branch file forward to that commit. Git writes no new object and computes no merge.

This is the one integration that adds nothing to history. It is available only when the branch never diverged.

Detached HEAD is one file holding a hash

It is not a mode, a state machine or an error. It is the contents of `.git/HEAD`.

```
$ cat .git/HEAD
ref: refs/heads/main
$ git checkout e043781
You are in 'detached HEAD' state.
$ cat .git/HEAD
e0437810144e64a708a6441a7094db56fe376eda
```

Commit now and Git writes a commit object, but no branch file moves. Switch away and nothing points at that commit any more.

The warning Git prints here is a description of that one file. It is not a diagnosis.

A merge is a function of three trees

A base, and two sides. Two versions are not enough to decide what happened.

With only two versions, "this line is in A and not in B" is ambiguous. It could be an addition on one side or a deletion on the other. The base resolves it.

Git compares each side against the base. Where one side changed a region, that side wins. Where both changed the same region, the function has no rule and emits a conflict.

A conflict is a line-range overlap

The criterion is textual. A merge that succeeds is not a merge that is correct.

Two people can edit different lines of the same function and produce a clean merge that does not compile. The comparison is over line ranges, and nothing in it depends on what the lines do.

The reverse also holds. Two people can edit the same line with the same intent, and the function still refuses, because the ranges overlap.

The base can be plural, and then one side is discarded

Two people resolve the same conflict in opposite directions. Now two commits qualify as the base.

Git names this a criss-cross. Both candidates are best common ancestors, and neither is better. `git merge-base` prints one of them and does not say which, while `git merge-base --all` prints both.

Picking either candidate makes the merge succeed cleanly and silently drops the other person's decision. The default merge strategy avoids that: it merges the two candidates into one synthesised base, and uses that as the third input.

Three ways to call the same function

The function is fixed. What changes is which inputs it gets and where its result is written.

```
printf 'base\n' > f.txt && git add f.txt && git commit -m base  
git checkout -b claire && printf 'claire\n' >> f.txt && git commit -am claire  
git checkout main && printf 'main\n' > g.txt && git add g.txt && git commit -m main
```

Both branches moved after base, so neither tip is an ancestor of the other. Nothing here can fast-forward.

git merge claire

One new commit with two parents.

git rebase main

Replay claire's commits onto main, one by one.

git merge --squash claire

One new commit with one parent.

Predict: which of the three leaves `c1a1r3` behind?

Three integrations of the same two branches. Write your answers down before anything runs.

- | | | |
|----------|-----------------|---|
| 1 | Content | Do all three put the same file content on <code>main</code> ? |
| 2 | Parents | How many parents does the new commit on <code>main</code> have, in each case? |
| 3 | Ancestry | Afterwards, is <code>c1a1r3</code> an ancestor of <code>main</code> ? |

Observe

```
$ git merge claire -m "merge claire"
Merge made by the 'ort' strategy.
$ git log -1 --format="parents=%p tree=%T"
parents=[bbd61c3 989d0bc] tree=ee2bb78e8e0f7f27012968f23574f2db7a95ef3b
$ git merge-base --is-ancestor claire HEAD && echo yes || echo no
yes

$ git checkout claire && git rebase main          # same branches, fresh repository
Successfully rebased and updated refs/heads/claire.
$ git log --oneline -2
a46813c claire          # was 989d0bc
bbd61c3 main

$ git merge --squash claire && git commit -m "squashed claire"
$ git log -1 --format="parents=%p tree=%T"
parents=[bbd61c3] tree=ee2bb78e8e0f7f27012968f23574f2db7a95ef3b
$ git merge-base --is-ancestor claire HEAD && echo yes || echo no
no
$ git merge-base main claire
b0f7638
```

Explain: one function, three placements

OPERATION	WHAT IT LEAVES IN THE GRAPH	PART I INVARIANT
Merge commit	Both tips become parents, so ancestry is preserved.	Adds objects
Rebase	One new commit per replayed commit, then the branch pointer moves.	Adds and moves
Squash	One new commit with one parent. The side branch is not an ancestor.	Adds objects

All three put the same content on main. Git's own merge and rebase manuals describe all three. They differ in what the graph then says about where that content came from.

Why a squash-merged branch keeps re-conflicting

The content arrived. The ancestry did not.

Merge-base discovery walks parent links. A squash commit has one parent, so the side branch's commits are not ancestors of anything on `main`.

Merge the same branch again and the base is computed from a graph in which that work never landed. Git re-offers changes that are already present, and they conflict with themselves.

A branch lands when its content reaches `main`. The fix is one line. Delete the branch once its content has landed. Start the next change from `main`.

Rebase keeps the tree and replaces the commit

A commit hashes its parents as well as its tree. Change the parent and you have a different object.

Replaying a commit onto a new base can produce exactly the same file content. The commit hash is still different, because the parent hash is part of what was hashed.

That is why every hash changes after a rebase. Rebasing work that other people already fetched forces them to reconcile two histories of the same changes.

Garbage collection deletes what no ref reaches

A rewrite abandons objects. Garbage collection is the later pass that removes them.

```
$ git reflog main
e043781 main@{1}: commit: First commit
9a1c4e0 main@{0}: rebase (finish): refs/heads/main onto origin/main
$ git branch rescue main@{1}
```

The reflog records every position a ref has held in this repository, and `main@{1}` names the position before the last one. Pointing a new branch at an old position makes the abandoned commits reachable again, so collection leaves them alone.

A remote is a name and a refspec

Not a connection and not a session. Two lines of configuration that `clone` writes for you.

```
$ git config --get remote.origin.url  
/tmp/demo.git  
$ git config --get remote.origin.fetch  
+refs/heads/*:refs/remotes/origin/*
```

The left side of the refspec names refs on the other repository, and the right side names refs in yours. The leading `+` allows the destination to be replaced even when that loses history, which is safe here because nothing of yours lives there. `origin` is simply the name `clone` chooses.

A bare repository is the usual other end. It holds the object store and the refs with no checked-out files.

origin/main is a local file

A third namespace of refs, stored on your disk, describing somebody else's repository.

```
$ cat .git/refs/remotes/origin/main  
93164b00faf155d30a8bcd8bc235f07f8e1bdc4
```

This is a remote-tracking ref. It records where the remote's branch was at your last fetch. It is not the remote, and it is not your branch.

"Up to date" is a claim about your last fetch

Nothing in `git status` reads the other repository.

```
$ git rev-parse origin/main          # reads a local file
93164b00faf155d30a8bced8bc235f07f8e1bdc4
$ git ls-remote origin refs/heads/main # reads the other repository
b31235e5902584aa5cea6f8f621cc41296e76406 refs/heads/main
$ git status -sb | head -1
## main...origin/main
```

The two commands disagree, and `status` follows the local file. It reported neither ahead nor behind while the other repository was one commit ahead.

fetch moves remote-tracking refs and nothing else

One command. One visible effect, and it is smaller than the name suggests.

```
≡ BEFORE ≡
local main:    93164b0
origin/main:   93164b0
worktree:      M f.txt
               ?? g.txt
## main...origin/main

≡ AFTER git fetch origin ≡
local main:    93164b0
origin/main:   b31235e
worktree:      M f.txt
               ?? g.txt
## main...origin/main [behind 1]
```

Exactly one line changed. The uncommitted edit survived, the untracked file survived, main did not budge. The count is right now because fetch refreshed the local ref that status reads.

pull refuses when the branches have diverged

Not a silent merge. With nothing configured, current Git stops and asks you to choose.

```
$ git config --get pull.rebase      # no output: unset
$ git config --get pull.ff         # no output: unset
$ git pull
hint: You have divergent branches and need to specify how to reconcile them.
hint:   git config pull.rebase false # merge
hint:   git config pull.rebase true  # rebase
hint:   git config pull.ff only      # fast-forward only
fatal: Need to specify how to reconcile divergent branches.
```

pull is a fetch followed by an integration step, and you choose the step. When the branches have not diverged, all three configured answers produce the same fast-forward.

Run the two `git config` queries first. A laptop where either setting already holds a value does not refuse.

push asks, and the remote applies one rule

The destination must be an ancestor of the commit you are sending.

The remote checks whether its current branch tip is an ancestor of your tip. If it is, moving the pointer forward loses no history. The remote then accepts the update.

If it is not, somebody else's commits are in the way. Moving the pointer would leave those commits unnamed, so the remote refuses.

Predict: a colleague pushed while you worked

One bare repository, two clones, one shared branch. Write down what each command changes.

```
git clone /tmp/demo.git a && git clone /tmp/demo.git b
# in b: commit and push one change
# in a: commit a different change on main, then:
```

- 1** **After git fetch** Which of main, origin/main and the working tree changes?
- 2** **After git pull** Does it complete, and what does it ask you?
- 3** **After git push** Accepted or refused, and by which rule?

Observe

```
$ git fetch
From /tmp/demo
   a210208..29aa22b  main      → origin/main
$ git status -sb | head -1
## main...origin/main [ahead 1, behind 1]
$ git pull
fatal: Need to specify how to reconcile divergent branches.
$ git push
! [rejected]        main → main (non-fast-forward)
error: failed to push some refs to '/tmp/demo.git'
hint: Updates were rejected because the tip of your current branch is behind
```

Fetch changed one ref. Pull asked a question. Push refused, and the message names the rule.

Explain: the refusal is a report

The remedy is to integrate and send a descendant, not to reach for a flag.

- 1 Read it**

Somebody else's commits are between your base and the remote tip.
- 2 Integrate**

Fetch, look at what arrived, then merge or rebase onto it.
- 3 Push again**

Your new tip now has the remote tip as an ancestor.

`--force-with-lease` sends the update only when the remote ref still holds the value your remote-tracking ref records. Fetching immediately before it weakens the check, because you then lease against a value you have not looked at.

Where centralisation went

Distribution removed a fact from the data model. It did not remove the need for it.

What a central server used to provide

One repository, on one server, and committing is the same act as publishing.

In a centralised system such as Subversion there is one repository, on one server. Committing sends the change to that server, so committing and publishing are the same act. Working offline is not possible for anything that needs history.

The server saw every operation, so it could also enforce two things. One was an exclusive lock on a file. The other was a rule about which paths a user may read.

Distribution removed the fact, not the need

Every clone is a complete repository. None of them is authoritative by construction.

Nothing in the object model marks one copy as the real one. Authority becomes a claim that people agree to honour, and the agreement is enforced somewhere outside the data model.

That somewhere is a forge: a hosting service that stores one repository and applies rules to it. Branch protection on a forge is doing the job the central server used to do structurally.

Locking needs a single arbiter

A lock is a claim over a resource, and a claim is only meaningful if one party decides it.

Two clones can each record that the lock is held, and nothing in a peer-to-peer model settles which record is right. A peer-to-peer data model therefore cannot provide locking. A team either relies on a social norm, or puts a single server back in the path.

The implication runs one way only. Locking requires centralisation. Centralisation does not require locking, which is why some centralised systems make locks optional.

Per-path read access has no Git equivalent

The unit of transfer is a reachable object graph, so there is no point at which a per-path rule applies.

A centralised server answers each command by consulting a table of paths and users. Git answers a fetch by sending the objects needed to complete a history.

To hide one directory you would have to rewrite the history being served. The workarounds therefore change the unit instead: split the tree into separate repositories, or enforce at the forge.

Git does ship enforcement, and it binds one repository

"Git has no access control" is too strong. Git ships hooks, and a hook can refuse a push.

Git ships hooks. A receiving repository can run an update hook that refuses any non-fast-forward push, or that consults a table before accepting a ref. Those are core Git, not forge features.

What Git does not do is choose the policy. And a hook binds the repository it lives in. Anyone who clones from you can bypass it entirely, and two peers who exchange commits directly are bound by nothing.

Can Git plus forge policy meet the requirement?

ONE AUTHORITATIVE HISTORY

GitHub
branch
protection

Yes. Protect the branch, and require every landing to be a fast-forward.

Policy

EXCLUSIVE LOCK ON A BINARY ASSET

—

No. A lock needs one arbiter that peers cannot provide.

Structural

READ ACCESS RESTRICTED PER PATH

—

No. Split the repository, or use a centralised system.

Structural

What Git costs, and who avoids paying it

Each limitation, paired with the system that made the opposite choice and what that choice cost.

Nothing in Git's design was the only option

Content addressing already existed, and a rival project began two weeks after Git.

Monotone stored objects by content hash before Git did. Torvalds tried it for a day and gave up on it, because its performance was so bad.

Mercurial's announcement is dated 20 April 2005. It states two aims: to be as simple as possible, and to be as scalable as possible.

So content addressing was not Git's invention. Git's properties are choices, and other projects took several of the alternatives.

The index has two roles, and only one needs to be visible

A cache of filesystem information, and a staging buffer the user operates.

The first role is uncontroversial. It is what makes `git status` fast enough to run constantly.

The second role is a design choice. Git exposes it, and exposes it under three different names across its own documentation: index, cache and staging area.

Keep the object graph, replace the layer above it

Jujutsu stores its data in a Git repository and discards Git's user model.

The working copy is itself a commit, snapshotted automatically. Because of that there is no index, and no stash either. A stash is Git's side pocket for unfinished edits, and an ordinary commit already does that job.

Everything the system changes sits strictly above the object graph, and it still interoperates with the same repositories. That is what makes it evidence about Git rather than a rival product.

It is pre-1.0, at release v0.45.1 of September 2026, and the project calls itself experimental. This is not migration advice.

cherry-pick writes a new object with no link to the original

Git derives changes by comparing snapshots. A change has no identity of its own.

Copying a commit to another branch builds a new commit object. Its content may be identical and its hash is different, because the parent differs. Nothing records that the two are the same change.

So Git cannot tell you that a fix is already present somewhere else, and a later merge can offer the same change again.

Darcs and Pijul make the change the object

Store patches with durable identities, and derive the tree by applying a set of them.

Once a change is a first-class object you can ask whether two changes commute. If A then B reaches the same state as B then A, the two are independent.

A change keeps its identity wherever it is applied, so cherry-picking is not a re-creation. A later merge recognises the patch rather than re-offering it.

The model is a real trade. Darcs still records its own exponential merge problem as unfixed, and Pijul has no stable release. No independent benchmark found in this research compares Pijul's merge speed with Git's.

Git stores no fact that one file became another

The rename is reconstructed at comparison time, from content similarity alone.

The design position was stated in 2005 and defended since. Git is about tracking data, and not about tracking what happened to create that data.

At comparison time Git scores each deleted path against each created path for content similarity. It merges the pair into a rename when the score passes a threshold. The default threshold is 50 percent.

A rename plus a large edit scores below the threshold

Put both in one commit and the trail of the file ends there.

```
$ git mv parser.c lexer.c
$ # ...then rewrite most of lexer.c in the same commit
$ git commit -qam 'rename and rewrite'
$ git log --follow --oneline lexer.c
9f2a1c8 rename and rewrite
```

The similarity score falls below 50 percent. Git reports a delete and an add, so `--follow` has nothing to cross.

The remedy is one commit per action. Commit the rename on its own. Commit the edit afterwards. No alternative system surveyed here removes the underlying cost.

Scale breaks one assumption, not the model

A commit is a whole-tree snapshot whether the tree holds fifty files or five million.

What breaks is an assumption layered on top of the model. The assumption is that one client can do three things cheaply:

- hold the whole tree
- transfer it on clone
- walk it on every status check

The production fixes split into two families. One fetches less, so the repository is genuinely incomplete. The other presents a complete working tree over an incomplete one. It fetches each file on demand.

A partial clone therefore gives up offline operation, because a missing object has to be fetched.

Trace one surprise to the choice that produced it

- 1** **Detached HEAD** HEAD may hold a raw hash, so a commit advances no branch. Jujutsu's working copy is itself a commit.
- 2** **A stash you lost** The stash is a side pocket outside history. Jujutsu keeps no index and no stash.
- 3** **A cherry-pick that conflicted again** A change carries no identity. Darcs and Pijul hash the change itself.

Name one Git behaviour that has cost you an hour. Then name the design choice that produces it, and the system that chose otherwise.

Branching strategy

Five named models, one variable underneath them, and the evidence for setting it low.

A branching strategy is a contract about branch lifetime

Git enforces none of this. A branch can sit unmerged for two years and no command refuses.

In these repositories the single shared branch is called `main`. The literature calls that branch the trunk.

A strategy says which branches exist and who may move them. It says how long a change may stay outside the trunk, and what it must pass on the way back. The team imposes every one of those constraints from outside the tool.

The named models differ mainly in that one variable, and almost every documented cost and benefit follows from it.

The five models you will meet

MODEL	LONG-LIVED BRANCHES	BUILT FOR
Trunk-based	main only, plus short release branches	Frequent delivery, high throughput
GitHub Flow	main only	Web applications that deploy on merge
GitLab Flow	main plus one branch per environment or version	Explicit promotion gates, several versions
Release Flow	main plus one branch per release	Several supported versions at large scale
Gitflow	main and develop, plus release and hotfix	Scheduled, explicitly versioned releases

Read the middle column. The difference between the five is which branches are allowed to live a long time, and that is the whole disagreement.

Three merge methods, three shapes of history

MERGE METHOD	WHAT LANDS ON THE TRUNK	AVAILABLE WHEN
Fast-forward	The branch commits, unchanged. No new object at all.	The branch never diverged
Merge commit	All branch commits, plus one commit with two parents.	Always
Squash	One new commit. Intermediate commits do not survive.	Always

A forge offers these as three buttons, and all three are ordinary Git operations.

Trunk-based development is a cadence, not a topology

The definition is stated in merges per day, not in the shape of the graph.

Each developer divides their work into small batches and merges into the trunk at least once a day, and potentially several times.

trunkbaseddevelopment.com defines it by what it forbids: collaborating in a single branch and resisting pressure to create other long-lived development branches. Short-lived branches for review and build checks are inside the model, not outside it.

Two variants, split by team size

Direct to trunk

Commit straight to the trunk. Checks run after the commit lands. Up to 15 people.

Short-lived branches

One branch per change, reviewed and checked before landing. Sixteen or more developers.

trunkbaseddevelopment.com puts the boundary at sixteen developers. It says the cut-off was 100 people before lightweight branching and the pull request, and is 15 now. Below the boundary a branch and a review for every change buy little, because few people commit. Above it, unverified commits land on the trunk and break it for everyone else.

Two preconditions that summaries leave out

2 days

the site's own risk threshold for becoming a long-lived branch

1 author

per branch, or two when pair programming

A branch that three people push to for a week is not a short-lived feature branch. It is a long-lived branch carrying a short-lived branch's name.

The trunk must also stay shippable, which is a testing property rather than a branching one. This is the healthy-branch rule: automated checks on each commit, so that the branch has no known defects.

Conflicts rise with divergence, and size predicts better than time

The largest mining study of merge conflicts, with the effect size that qualifies it.

One study covers 182,273 merge scenarios in 80 projects. The comparison is against the study's own average conflict rate. Branches integrated in under a day produced about 40% fewer conflicts. Branches older than 30 days produced about 161% more.

The same paper reports the effect size for branch duration as small, at minus 0.3 on its own scale. It finds the number of changed files, commits and authors to predict conflicts more strongly. So size is the stronger predictor, and duration is largely standing in for it.

The resolutions themselves ship defects

A conflict is not merely annoying. The code that comes out of one is measurably worse.

Across 143 open-source projects and 36,122 merge commits, 19.32% produced conflicts that a person had to resolve.

Lines involved in a conflict were about twice as likely to be buggy as lines that were not. Lines involved in a conflict of entangled syntax-tree changes were about 26 times more likely to need a bug-fixing commit than lines from trivial conflicts.

The delivery evidence points the same way, and is weaker

Two lines of evidence with different weaknesses. One objection does not remove both.

The survey research associates higher delivery performance with teams that keep three or fewer active branches and merge to the trunk at least daily.

It is self-reported and correlational. Nothing in the data establishes that adopting the practice causes the performance rather than accompanying the test automation that makes it survivable.

Nobody defends long-lived feature branches

The argument that survives is about release branches, and whether your product needs them.

Vincent Driessen published the best-known model in 2010, and it is usually called Gitflow. He narrowed it himself in 2020. For a team that ships on demand he suggests a much simpler workflow. For software that is explicitly versioned, or that supports multiple versions in the wild, he says his model may still fit.

Microsoft's documented practice does both at once. Short-lived branches come off `main`. The team cuts a release branch for each three-week iteration. That branch never merges back, and fixes reach it by copying a commit forward from `main`.

Flags replace branches, and the debt is measured

Unfinished work has to be isolated somewhere. A branch isolates it in history, a flag isolates it at run time.

An unmerged branch is listed by `git branch` until somebody deals with it. A feature flag is a switch in the code that hides unfinished work from the user. A flag left in place after a release is listed nowhere.

Across 100 projects holding 7,593 flags, half were removed within fifteen months, and a quarter remained for more than eight years. Accumulation is the default path, not an occasional lapse.

Which branching strategy does your project earn?

- 1** **Can checks run in minutes?** If not, short-lived branches deliver instability rather than speed.
- 2** **One author per branch?** A shared branch is a long-lived branch with a short name.
- 3** **Versions live in the field?** A customer on an old version needs a patch. That earns a release branch.
- 4** **How big is the change?** Duration is largely a proxy for size. Ask the size too.

Answer these four about your own group project. Two answers of no mean short-lived branches will not work yet.

Pull requests and review

Not Git. A convention roughly eighteen years old, and what the measurements actually say about it.

The name is from 2005, the button is from 2011

A pull request was originally a request that somebody else perform a pull.

A maintainer with work ready mailed a message naming a repository and a list of commits, and asked the recipient to fetch it. Git shipped a command to generate exactly that message in July 2005.

In February 2008 GitHub wrapped that message in a web page. Inline comments on the difference arrived in February 2011, three years later. The button that performs the merge arrived in April 2011, two months after that. Until then every pull request was merged by hand.

What reviewers write about is mostly not defects

One study classified 570 comments from 200 review threads at a large software company.

Finding defects was the top-ranked motivation the participants gave. In the classified comments, defect was the fourth most frequent category of nine, at 14%. Code improvements was the most frequent, at 29%.

The study's own summary says review comments about defects are few, at roughly one eighth of the sample. It adds that they mostly address superficial concerns rather than design-level ones.

That does not make review idle. A separate study still estimates up to two and five extra defects after release where review coverage and participation are low.

The outcome evidence cuts the other way

Counting what reviewers write is not the same as measuring what review prevents.

The two-and-five estimate came from measuring components rather than comments. It set how much review each component received against how many defects that component showed after release.

So the honest claim is narrower than either headline. Review spreads understanding of the change: afterwards a second person can alter that code without asking the author. It also catches some defects, and only up to a size limit.

Review is comprehension, and comprehension is the bottleneck

A reviewer does not fail to find a defect through carelessness. Finding one requires understanding the change.

The same study found that code and change understanding is the key aspect of reviewing. It also found that 798 of 873 surveyed programmers confirmed that unfamiliar files take longer.

This is the mechanism underneath every rule about small changes. Small changes are not tidier. They are comprehensible.

Two numbers, one source, and they are not in conflict

100–300

lines in 30 to 60 minutes, the authors' own advice

400

lines, the ceiling, because more overwhelms a reviewer

A ten-month study of 2,500 reviews covering 3.2 million lines produced both figures. The first pairs a size with a time. The second is a size limit on its own.

Time carries a separate finding in the same study. Reviewers found defects at a roughly constant rate through the first 60 minutes, after which the rate levelled off. No reviewer found a defect after 90 minutes.

A branch is old because it is waiting

Review latency does not merely delay a change. It grows the change it then fails to review.

A change sits waiting for a reviewer. The author has more work that depends on it, so the commits go onto the same branch rather than a new one.

The change that eventually reaches a reviewer is therefore larger than the one that was submitted. Past 400 lines the review measurement found no further defects, so the waiting produces the size that review can no longer absorb.

Git cannot record that one branch depends on another

A commit records its parents. Nothing records its children.

A stack is one change split into a chain of branches, each built on the branch below it. There is no object that says branch B is stacked on branch A. Nothing in the repository records that editing the bottom of a chain invalidates every layer above it.

Every tool that offers stacked review closes that gap the same way: by keeping each branch's intended parent outside the commit graph. Every such tool therefore has a state that can disagree with the repository.

Two operations, under many names

Retarget

Point each change's comparison base at the change below it, so its difference shows only its own layer.

Restack

After any edit to a lower layer, replay every layer above it and move the intermediate branch tips.

Interactive rebase has carried the second operation since Git 2.38, as `git rebase -i --update-refs`. On 30 July 2026 GitHub put stacked pull requests into public preview, which is a trial release rather than the finished feature.

Squash-merging one layer strands the layers above it

The Part II squash result, applied to a chain.

Squash the bottom layer and the trunk gains one commit with one parent. The original branch is not an ancestor of it.

The layer above still sits on the old version of the layer below. Its merge base falls back to the root, and it re-presents content that is already integrated.

Review when nobody wrote the code

The measured output of review assumes a person wrote the change and a person absorbs the comment.

If a language model produced the change, nobody on the team ever held the reasoning behind it, so there is no understanding to transfer. Catching defects is then the only function review has left, and the measured size and time limits bound that function.

The measurement to watch is where the effort goes. In one 2026 developer survey the work moved from writing code to verifying it, rather than disappearing. That is an open question, not a settled result.

One pull request, or a stack?

- | | | |
|----------|-------------------------|---|
| 1 | Under 300 lines | One change. A stack costs coordination it does not need. |
| 2 | Separable layers | Stack it, so each layer is reviewed against the one below. |
| 3 | Landing a stack | Merge bottom upwards. Squash an intermediate layer and the rest re-conflicts. |

Integration, delivery, deployment

Three different practices, one abbreviation, and a branching strategy that depends on which one you do.

Three practices wearing one abbreviation

Continuous integration

A property of how a team uses version control.

Continuous delivery

A property of the artefact the pipeline produces.

Continuous deployment

A policy about what happens to an artefact that passes.

The pipeline is the automated sequence of builds and tests that every change goes through.

Each one requires the one before it. A team can integrate continuously and release twice a year. A team cannot deploy continuously without first being able to deliver continuously.

Continuous integration is a frequency claim

It is about how often people merge, not about which server runs the build.

Each member merges their changes into the shared codebase together with their colleagues' changes at least daily. Each of those integrations is verified by an automated build that includes tests.

Two clauses, both routinely dropped. At least daily makes it a claim about cadence. Verified by an automated build including tests makes it a claim about a self-testing build.

Three questions that settle it

- 1** **Daily** Is everyone pushing to the trunk every day, rather than to feature branches?
- 2** **Every commit** Does every commit trigger a run of the unit tests?
- 3** **Ten minutes** When the build breaks, is it typically fixed within ten minutes?

The author of the test reports that in his experience fewer than 20% of teams who believe they are doing this can pass it. That figure is one practitioner's recollection, not a measurement.

The same practice under two names

The branching choice in Part V and the integration practice here are one decision.

The author of the canonical definition writes that this practice is called continuous integration, or in some circles trunk-based development.

That is why the preconditions match. Merging daily requires a trunk that stays shippable, which requires automated checks on every commit, which is the healthy-branch rule from Part V.

Continuous delivery is a property of the artefact

The team builds the software so that it can go to production at any time. Production is the running system users depend on, and nobody has to release to it.

The ThoughtWorks continuous delivery working group reduced it to four observable indicators.

- The software is deployable throughout its life.
- The team prioritises keeping it deployable over new features.
- Anyone can get fast automated feedback on production readiness.
- Any version deploys to any environment on demand.

The acceptance test is social. A sponsor could ask for the current development version to go to production at a moment's notice, and nobody would panic.

Continuous deployment is a policy

Every change that passes the pipeline goes to production automatically.

Nothing about the artefact changes. What changes is who decides to release it, and the answer becomes nobody.

That is why it sits last in the dependency order. Releasing an artefact the team does not trust automatically reaches an outage faster than it reaches delivery performance.

Feature flags follow from the release policy

Two sources disagree about whether flags are required. The disagreement dissolves along the delivery and deployment line.

The goal is to separate the technical decision to deploy from the business decision to launch a feature. Where releasing is automatic, the team must hide unfinished work at run time. That needs a feature flag, or a dark launch, which ships the code with nothing calling it yet.

Where a person still picks the release moment, that person is the gate, and the flag is optional. The author of the pattern says exactly this: flags are only really necessary when practising continuous deployment.

Deployment frequency stands in for batch size

Batch size is how much change ships at once. It cannot be counted directly, so the number of releases stands in for it.

A smaller change fits inside the 400-line review limit from Part VI, and reverting it removes less than reverting a large one. Reducing the cost of a deployment is what makes smaller batches affordable at all.

The proxy is under strain. The 2024 survey report states that it is possible, even likely, that change sizes are growing, and it did not measure change size. Treat the proxy as a proxy.

Diagnose what a team actually practises

WHAT TO LOOK AT	THE QUESTION	WHICH PRACTICE
Merge cadence	Daily to the trunk, or branches that live for weeks?	Integration
The artefact	Could any commit that passed the pipeline ship now?	Delivery
The trigger	Does passing the pipeline release, or does a person decide?	Deployment

Fowler's definitions supply all three questions. A team that answers no to the first is not doing any of the three, whatever its pipeline is called.

Takeaways, restated

- | | | |
|----------|----------------------------------|--|
| 1 | Objects and pointers | A commit, rebuilt by hand. A branch is a file holding a hash. |
| 2 | One merge function | A squash kept the tree and lost the ancestry. |
| 3 | Trade-offs, not laws | Locking, per-path access, patch identity. Each has a rival design. |
| 4 | Branch lifetime | Two days, one author, checks on every commit. |
| 5 | What review produces | Understanding first, defects second, none found after 90 minutes. |
| 6 | Three practices, one name | Cadence, artefact, trigger. Ask all three. |

What follows from the object graph

The thesis, stated again now that every part of it has been shown.

Git gives you a content-addressed object graph and a way to move pointers into it. The object model has been stable since 2005.

Branching strategy, pull requests, code review and delivery practice are policy your team layered on top. They are choices, they are contested, and the evidence for them is weaker than the evidence for the graph. The honest limit: Git does ship hooks, so it supplies the enforcement mechanism and sets no policy.

What this course leaves open

- **Forge-specific work** Part VI
The mechanism and the research, not one vendor's interface or its merge queue.
- **The wider remote surface** Part II
Multiple remotes, forks, custom refsspecs, and configuring the integration step.
- **History rewriting** Part II
Interactive rebase corner cases, filtering a repository, submodules, work trees.
- **Spending the graph** Part II
`git bisect` searches the ancestry you now understand. It was not covered here.

Where to read next

- 1** **The object model** Pro Git, chapter 10. The plumbing sequence from Part I, in full.
- 2** **The short version** The MIT Missing Semester lecture. One hour, and it states the model as type declarations.
- 3** **The graph, visually** Learn Git Branching. A browser sandbox that draws the commit graph.
- 4** **Why it is awkward** A conceptual-design analysis of Git's interface. Read it once the object model is in place.

Voluntary homework comes in three steps. Rebuild a commit by hand. Write all three predictions down. Then run the Part II demonstration.

References • 1/16

- **[S1]** Pro Git — Git Internals: Git Objects — Chacon & Straub. <https://git-scm.com/book/en/v2/Git-Internals-Git-Objects>
- **[S2]** Pro Git — Git Internals: Plumbing and Porcelain — Chacon & Straub. <https://git-scm.com/book/en/v2/Git-Internals-Plumbing-and-Porcelain>
- **[S3]** Pro Git — Git Internals: Git References — Chacon & Straub. <https://git-scm.com/book/en/v2/Git-Internals-Git-References>
- **[S4]** Pro Git — What is Git? — Chacon & Straub. <https://git-scm.com/book/en/v2/Getting-Started-What-is-Git%3F>
- **[S5]** Pro Git — Git Internals: Packfiles — Chacon & Straub. <https://git-scm.com/book/en/v2/Git-Internals-Packfiles>

References · 2/16

- **[S6]** gitglossary(7) — The Git project. <https://git-scm.com/docs/gitglossary>
- **[S7]** git-merge-base(1) — The Git project. <https://git-scm.com/docs/git-merge-base>
- **[S8]** git-merge(1) — The Git project. <https://git-scm.com/docs/git-merge>
- **[S9]** git-rebase(1) — The Git project. <https://git-scm.com/docs/git-rebase>
- **[S10]** git-push(1) — The Git project. <https://git-scm.com/docs/git-push>
- **[S11]** git-fetch(1) — The Git project. <https://git-scm.com/docs/git-fetch>
- **[S12]** git-pull(1) — The Git project. <https://git-scm.com/docs/git-pull>
- **[S13]** gitattributes(5) — The Git project. <https://git-scm.com/docs/gitattributes>
- **[S14]** githooks(5) — The Git project. <https://git-scm.com/docs/githooks>

References · 3/16

- **[S15]** BreakingChanges — The Git project. <https://git-scm.com/docs/BreakingChanges>
- **[S16]** Git 3.0 gets a date — Corbet, LWN (2026).
<https://lwn.net/SubscriberLink/1094575/2385e98583715c2b/>
- **[S17]** Version Control (Git) — MIT Missing Semester. <https://missing.csail.mit.edu/2020/version-control/>
- **[S18]** Re: Handling renames — Torvalds, git mailing list (2005). <https://public-inbox.org/git/Pine.LNX.4.58.0504141102430.7211@ppc970.osdl.org/>
- **[S19]** Google Tech Talk transcript — Torvalds (2007).
<https://archive.kernel.org/oldwiki/git.wiki.kernel.org/index.php/LinusTalk200705Transcript.html>

References • 4/16

- **[S20]** Making sense of distributed version control — Isomottonen & Cochez.
<https://www.cochez.nl/papers/dvcs-misconceptions-final.pdf>
- **[S21]** What's Wrong with Git? A Conceptual Design Analysis — Perez De Rosso & Jackson (2013).
<https://groups.csail.mit.edu/sdg/pubs/2013/git-conceptual-onward13.pdf>
- **[S22]** Jujutsu — Git comparison — The Jujutsu project. <https://docs.jj-vcs.dev/latest/git-comparison/>
- **[S23]** Expectations, Outcomes, and Challenges of Modern Code Review — Bacchelli & Bird (2013).
<https://www.microsoft.com/en-us/research/wp-content/uploads/2016/02/ICSE202013-codereview.pdf>
- **[S24]** The Impact of Code Review Coverage and Participation on Software Quality — McIntosh, Kamei, Adams & Hassan (2014). https://rebels.cs.uwaterloo.ca/papers/msr2014_mcintosh.pdf

References • 5/16

- **[S25]** Best Kept Secrets of Peer Code Review — Cohen, Teleki & Brown (2006). <https://static1.smartbear.co/support/media/resources/cc/book/best-kept-secrets-of-peer-code-review.pdf>
- **[S26]** Trunk Based Development — Introduction — trunkbaseddevelopment.com. <https://trunkbaseddevelopment.com/>
- **[S27]** Short-Lived Feature Branches — trunkbaseddevelopment.com. <https://trunkbaseddevelopment.com/short-lived-feature-branches/>
- **[S28]** Capabilities: Trunk-based development — DORA. <https://dora.dev/capabilities/trunk-based-development/>
- **[S29]** A successful Git branching model — Driessen (2010, revised 2020). <https://nvie.com/posts/a-successful-git-branching-model/>

References • 6/16

- **[S30]** Patterns for Managing Source Code Branches — Fowler (2020).
<https://martinfowler.com/articles/branching-patterns.html>
- **[S31]** Continuous Integration — Fowler (2023).
<https://martinfowler.com/articles/continuousIntegration.html>
- **[S32]** Continuous Delivery — Fowler. <https://martinfowler.com/bliki/ContinuousDelivery.html>
- **[S33]** Patterns — Continuous Delivery — Humble.
<https://continuousdelivery.com/implementing/patterns/>
- **[S34]** Continuous Integration — foundations — Humble.
<https://continuousdelivery.com/foundations/continuous-integration/>

References • 7/16

- **[S35]** How Microsoft develops with DevOps — Microsoft. <https://learn.microsoft.com/en-us/devops/develop/how-microsoft-develops-devops>
- **[S36]** About pull request merges — GitHub Docs. <https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/incorporating-changes-from-a-pull-request/about-pull-request-merges>
- **[S37]** Stacked pull requests are now in public preview — GitHub Changelog (2026). <https://github.blog/changelog/2026-07-30-stacked-pull-requests-are-now-in-public-preview/>
- **[S38]** Attributes that may raise the occurrence of merge conflicts — Menezes et al. (2021). <https://doi.org/10.5753/jserd.2021.1911>
- **[S39]** An empirical investigation into merge conflicts and their effect on software quality — Brindescu et al. (2020). https://stairs.ics.uci.edu/papers/2020/emperical_MC.pdf

References · 8/16

- **[S40]** Capture the Feature Flag: Detecting Feature Flags in Open-Source — Meinicke et al. (2020). <https://cmustrudel.github.io/papers/msr20flags.pdf>
- **[S41]** Darcs — Frequently Asked Questions (Performance) — The Darcs project. <https://darcs.net/FAQ/Performance>
- **[S42]** Mercurial v0.1 — a minimal scalable distributed SCM — Mackall (2005). <https://lkml.iu.edu/hypermail/linux/kernel/0504.2/0670.html>
- **[S43]** Predict-Observe-Explain — Taber. <https://science-education-research.com/teaching-science/constructivist-pedagogy/predict-observe-explain/>
- **[S44]** About protected branches — GitHub Docs. <https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches/about-protected-branches>

References · 9/16

- **[S45]** pcottle/learnGitBranching — Peter Cottle. <https://github.com/pcottle/learnGitBranching>
- **[S46]** gitdiffcore — Tweaking diff output — Git documentation. <https://git-scm.com/docs/gitdiffcore>
- **[S47]** Git 2.38 Release Notes — Git project (2022).
<https://github.com/git/git/blob/v2.38.0/Documentation/RelNotes/2.38.0.txt>
- **[V1]** Git — Origin, Object Model, and Design Trade-offs — vault note.
- **[V2]** Git — Origin, Object Model, and Design Trade-offs — vault note.
- **[V3]** Git — Origin, Object Model, and Design Trade-offs — vault note.
- **[V4]** Git — Origin, Object Model, and Design Trade-offs — vault note.

References • 10/16

- **[V5]** Git — Origin, Object Model, and Design Trade-offs — vault note.
- **[V6]** Git — Origin, Object Model, and Design Trade-offs — vault note.
- **[V7]** Git — Origin, Object Model, and Design Trade-offs — vault note.
- **[V8]** Git — Origin, Object Model, and Design Trade-offs — vault note.
- **[V9]** Git — Origin, Object Model, and Design Trade-offs — vault note.
- **[V10]** Git — Origin, Object Model, and Design Trade-offs — vault note.
- **[V11]** Merge Algorithms — Three-Way Merge, Rebase, and First-Class Conflicts — vault note.
- **[V12]** Merge Algorithms — Three-Way Merge, Rebase, and First-Class Conflicts — vault note.
- **[V13]** Merge Algorithms — Three-Way Merge, Rebase, and First-Class Conflicts — vault note.

References · 11/16

- **[V14]** Merge Algorithms — Three-Way Merge, Rebase, and First-Class Conflicts — vault note.
- **[V15]** Merge Algorithms — Three-Way Merge, Rebase, and First-Class Conflicts — vault note.
- **[V16]** Remotes, Remote-Tracking Refs, and Refspecs — What Fetch, Push, and Pull Actually Do — vault note.
- **[V17]** Remotes, Remote-Tracking Refs, and Refspecs — What Fetch, Push, and Pull Actually Do — vault note.
- **[V18]** Remotes, Remote-Tracking Refs, and Refspecs — What Fetch, Push, and Pull Actually Do — vault note.
- **[V19]** Remotes, Remote-Tracking Refs, and Refspecs — What Fetch, Push, and Pull Actually Do — vault note.

References · 12/16

- **[V20]** Remotes, Remote-Tracking Refs, and Refspecs — What Fetch, Push, and Pull Actually Do — vault note.
- **[V21]** Remotes, Remote-Tracking Refs, and Refspecs — What Fetch, Push, and Pull Actually Do — vault note.
- **[V22]** Distribution moves centralisation from the data model into policy — vault note.
- **[V23]** Distribution moves centralisation from the data model into policy — vault note.
- **[V24]** Distribution moves centralisation from the data model into policy — vault note.
- **[V25]** Distribution moves centralisation from the data model into policy — vault note.
- **[V26]** Jujutsu — A Redesign of the Git User Model — vault note.

References · 13/16

- **[V27]** Jujutsu — A Redesign of the Git User Model — vault note.
- **[V28]** Patch Theory — Darcs, Pijul, and Commuting Changes — vault note.
- **[V29]** Patch Theory — Darcs, Pijul, and Commuting Changes — vault note.
- **[V30]** Patch Theory — Darcs, Pijul, and Commuting Changes — vault note.
- **[V31]** Git's Conceptual Design Misfits — the Gitless Critique — vault note.
- **[V32]** Every Git Scaling Fix Either Fetches Less or Fakes the Rest — vault note.
- **[V33]** Branching Strategies — Trunk-Based vs Branch-Based Development — vault note.
- **[V34]** Branching Strategies — Trunk-Based vs Branch-Based Development — vault note.
- **[V35]** Branching Strategies — Trunk-Based vs Branch-Based Development — vault note.

References · 14/16

- **[V36]** Branching Strategies — Trunk-Based vs Branch-Based Development — vault note.
- **[V37]** Branching Strategies — Trunk-Based vs Branch-Based Development — vault note.
- **[V38]** Code Review and the Pull Request as a Review Mechanism — vault note.
- **[V39]** Code Review and the Pull Request as a Review Mechanism — vault note.
- **[V40]** Code Review and the Pull Request as a Review Mechanism — vault note.
- **[V41]** Code Review and the Pull Request as a Review Mechanism — vault note.
- **[V42]** Stacked diffs — reviewing a change as a series of small dependent changes — vault note.
- **[V43]** Stacked diffs — reviewing a change as a series of small dependent changes — vault note.
- **[V44]** Continuous Integration and Continuous Delivery — vault note.

References • 15/16

- **[V45]** Continuous Integration and Continuous Delivery — vault note.
- **[V46]** DORA Metrics — vault note.
- **[V47]** Deployment Frequency — vault note.
- **[V48]** DORA Metrics Limitations in AI-Assisted Development — vault note.
- **[V49]** The Small Batches Principle — vault note.
- **[V50]** Copy-on-Write — vault note.
- **[V51]** Code Review and the Pull Request as a Review Mechanism — vault note.
- **[V52]** Code Review and the Pull Request as a Review Mechanism — vault note.
- **[V60]** Code Review and the Pull Request as a Review Mechanism — vault note.

References · 16/16

- **[V61]** Code Review and the Pull Request as a Review Mechanism — vault note.
- **[V62]** Git — Origin, Object Model, and Design Trade-offs — vault note.
- **[V63]** Git — Origin, Object Model, and Design Trade-offs — vault note.

The references document, as a PDF and a web page, carries the full source list and a summary of every vault note with its own external references.

TAKE IT WITH YOU

Everything from this talk

[Slides \(PDF\)](#)

[References \(PDF\)](#)

[References \(web page\)](#)

[Speaker notes \(PDF\)](#)

[Everything \(.zip\) — readable by an AI agent](#)

ondrasek.github.io/talks

me@ondrejkrasicek.com